

1 TypeSpecs Syntax Specification

Annotations	A ::= $\overline{\text{@spec } E}$
Expressions	E ::= $x(\bar{T}) :: T \mid x(\bar{T}) :: T \text{ when } \overline{\alpha : \bar{T}}$
Types	T ::= $\alpha \mid T \mid T$ $\mid \text{any}() \mid \text{none}()$ $\mid \text{atom}() \mid :k$ $\mid \text{pid}() \mid \text{port}() \mid \text{reference}()$ $\mid \text{float}() \mid \text{integer}()$ $\mid \text{neg_integer}() \mid \text{non_neg_integer}() \mid \text{pos_integer}()$ $\mid \text{tuple}() \mid \{\bar{T}\} \mid \% \{\bar{F}\}$ $\mid [] \mid \text{nonempty_maybe_improper_list}(T, T)$ $\mid \ll \mid \ll_::n \gg \mid \ll_::_ * n \gg \mid \ll_::n, _::_ * n \gg$ $\mid (\bar{T} \rightarrow T) \mid (\dots \rightarrow T)$ $\mid n \mid n..n'$
Fields	F ::= $\text{required}(T) \Rightarrow T \mid \text{optional}(T) \Rightarrow T$

(where x ranges over expression variables, α ranges over type variables, n ranges over integers with $n < n'$, and k ranges over atom names.)

Figure 1: TypeSpec

It is noteworthy that the notation $\overline{E}$ denotes a possibly empty sequence of elements E separated by commas apart from $\overline{\alpha : \bar{T}}$, where it denotes a non-empty sequence. Furthermore, not only is $\text{@spec } \overline{E}$ non-empty sequential elements but also multi-line elements unlike the other comma-separated sequences. That is TypeSpecs only allows one specification on one line.

This specification addresses the most fundamental and atomic types based on the TypeSpecs reference on Elixir documentation. The addressed are, starting from the top type **any()** and the bottom type **none()**, controversially vague concept of field types of record types. While thus enables expressing every existing TypeSpecs type, some might come up with questions on fundamentality with subtype relations or notion of expressions between basic types and literal types. In the following sections, these questions are addressed for clarity with listing the built-in types and the syntactic sugars in Table 1 and Table 2.

1.1 TypeSpecs Basic Types and Literals

TypeSpecs is rather expressive especially by providing the literal types that are also used in the code with the same form. However, this is classic that the higher expressiveness brings the higher complexity and TypeSpecs is not exempt. The common trade-off must be discussed here due to the fact that our ultimate goal is to translate TypeSpecs to Elixir Types. In the perspective of translation, while TypeSpecs allows literals types such as $:k$, $[]$, $\ll$, $(\bar{T} \rightarrow T)$ and $n..n'$, Elixir Types will not support $\ll$, and the range type (i.e., $n..n'$) will not appear to users but will be only used internally. This difference will not allow direct translations for all the types and eventually necessitate approximations of them in Elixir Types.

The first question could arise is from the subtype relationship between **atom()** and $:k$. It is true that $:k$ is a subtype of **atom()** since $:k$ represents every single atom type and **atom()** represents a set of all atom types. Nonetheless, we have to contain $:k$ in the syntax specification. That is because $:k$ is a literal type that could be any single atom type, e.g., **true** and **false** are literals that are used as themselves in the code and also in type specification. Therefore, it is always sound for **atom()** to replace **true** or **false** but not vice versa.

It could be viewed that the range literal (i.e., $n..n'$) expresses **neg_integer()**, **non_neg_integer()** and **pos_integer()**. This idea could be backed by Types and Their Syntax in Erlang documentation that these are built-in types which can be thought defined by $..-1$, $0..$ and $1..$, respectively. But Elixir documentation specifies them as basic types and the provided range syntax in Erlang documentation is not correct use. Provided, it is sufficient to say that these are not subtypes of $n..n'$, hence they are necessary to be in the syntax specification.

1.2 TypeSpecs Built-in Types and Syntactic Sugar

Built-in types	Defined as
term()	any()
arity()	0..255
*as_boolean(T)	T
binary()	«_::_*8»
nonempty_binary()	«_::_8, _::_*8»
bitstring()	«_::_*1»
nonempty_bitstring()	«_::_1, _::_*1»
boolean()	:true :false
byte()	0..255
nonempty_list(T)	nonempty_maybe_improper_list(T, [])
list(T)	[] nonempty_list(T)
nonempty_list()	nonempty_list(any())
*nonempty_improper_list(T ₁ , T ₂)	nonempty_maybe_improper_list(T ₁ , T ₂)
maybe_improper_list(T ₁ , T ₂)	[] nonempty_improper_list(T ₁ , T ₂)
maybe_improper_list()	maybe_improper_list(any(), any())
nonempty_maybe_improper_list()	nonempty_maybe_improper_list(any(), any())
char()	0..0x10FFFF
charlist()	list(char())
nonempty_charlist()	nonempty_list(char())
fun()	(... -> any())
function()	fun()
identifier()	pid() port() reference()
iodata()	iolist() binary()
iolist()	maybe_improper_list(byte() binary() iolist(), binary() [])
keyword()	[{atom(), any()}]
keyword(T)	[{atom(), T}]
mfa()	{module(), atom(), arity()}
module()	atom()
no_return()	none()
node()	atom()
number()	integer() float()
struct()	%{:_struct_ => atom(), optional(atom()) => any()}
timeout()	:infinity non_neg_integer()

(built-in types with prepending asterisk (*) are only listed for documentation)

Table 1: TypeSpecs Built-in Types

Syntactic sugar	Expanded as
true	:true
false	:false
nil	:nil
[T, ...]	nonempty_list(T)
[T]	list(T)
T ₁ => T ₂	required(T ₁) => T ₂
k:T	:k => T
%k{F}	%{:__struct__ => :k, F}

Table 2: TypeSpecs Syntactic Sugars

2 Elixir Types Syntax Specification

Annotations	A ::= \$S \$S when $\overline{\alpha:T}$	
Signatures	S ::= $\overline{T} \rightarrow T$ S and S	
Types	T ::= α K term() none() dynamic() T or T T and T not T n..n infty..n n..infty { $\overline{T}$ } { $\overline{T}, \dots$ } %{ $\overline{F}$ } %{ $\dots, \overline{F}$ } ($\overline{T} \rightarrow T$) empty_list() non_empty_list(T, T)	type variables basic types type connectives integers (internal representation) type constructors
Key types	K ::= atom() :k pid() port() reference() float() integer() bitstring() binary() tuple() open_map() fun() list()	
Fields	F ::= :k=>T K=>if_set(T)	

where x ranges over expression variables, α ranges over type variables, n ranges over integers (integers internal representation: types not available to the programmer), and k ranges over atom names. (N.B., the notation $\overline{E}$ denotes a possibly empty sequence of elements E separated by commas apart from { $\overline{T}, \dots$ }, %{ $\dots, \overline{F}$ }, and $\overline{\alpha:T}$, where it denotes a non-empty sequence. Also, in $\overline{F}$ each F must have a different keytype)

Figure 2: Elixir Types Syntax

2.1 Elixir Type Built-in Types and Syntactic Sugar

Built-in types	Defined as
<code>boolean()</code>	<code>:true</code> or <code>:false</code>
<code>number()</code>	<code>integer()</code> or <code>float()</code>
<code>non_empty_list(T)</code>	<code>non_empty_list(T, empty_list())</code>
<code>list(T, T')</code>	<code>empty_list()</code> or <code>non_empty_list(T, T')</code>
<code>list(T)</code>	<code>empty_list()</code> or <code>non_empty_list(T, empty_list())</code>
<code>list()</code>	<code>empty_list()</code> or <code>non_empty_list(term(), empty_list())</code>

Table 3: Elixir Type Built-in Types

Syntactic sugar	Expanded as
<code>{...}</code>	<code>tuple()</code>
<code>%{...}</code>	<code>open_map()</code>
<code>k : T</code>	<code>:k => T</code>
<code>K => not_set()</code>	<code>K => if_set(none())</code>
<code>K => T</code> <i>(for $K \neq :k$)</i>	<code>K => if_set(T)</code>

Table 4: Elixir Type Syntactic Sugars

3 Translation from TypeSpecs to Elixir Types

$\llbracket \cdot \rrbracket : \text{TypeSpecs} \rightarrow \text{Elixir type}$

Figure 3: Semantic of Translation

3.1 Translation of TypeSpecs Specification into Elixir Annotations

$$\begin{aligned}
\llbracket @spec\ E \rrbracket &= \$ \llbracket E \rrbracket \\
\llbracket @spec\ \bar{E}, @spec\ E' \rrbracket &= \$ \llbracket \bar{E}, E' \rrbracket \\
\llbracket x(\bar{T}_1) :: T_2 \rrbracket &= \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \\
\llbracket x(\bar{T}_1) :: T_2 \text{ when } \bar{\alpha} : \bar{T}_3 \rrbracket &= \llbracket \bar{T}_1 \rrbracket \rightarrow \llbracket T_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\
\llbracket \bar{E}, x(\bar{T}'_1) :: T'_2 \rrbracket &= \begin{cases} S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ \llbracket \bar{E} \rrbracket \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket & \text{otherwise} \end{cases} \\
\llbracket \bar{E}, x(\bar{T}'_1) :: T'_2 \text{ when } \bar{\alpha}' : \bar{T}'_3 \rrbracket &= \begin{cases} S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket, \bar{\alpha}' : \llbracket \bar{T}'_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ & \text{and } \bar{\alpha} \cap \bar{\alpha}' = \emptyset \\ S \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket [\alpha' := \beta'] \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket, \bar{\beta}' : \llbracket \bar{T}'_3 \rrbracket & \text{if } \llbracket \bar{E} \rrbracket \approx S \text{ when } \bar{\alpha} : \llbracket \bar{T}_3 \rrbracket \\ & \text{and } \bar{\alpha} \cap \bar{\alpha}' \neq \emptyset \\ \llbracket \bar{E} \rrbracket \text{ and } \llbracket \bar{T}'_1 \rrbracket \rightarrow \llbracket T'_2 \rrbracket \text{ when } \bar{\alpha}' : \llbracket \bar{T}'_3 \rrbracket & \text{otherwise} \end{cases}
\end{aligned}$$

Figure 4: Translation into Annotations

3.2 Translation of TypeSpecs into Elixir Types

$$\begin{aligned}
\llbracket \alpha \rrbracket &= \alpha \\
\llbracket T_1 \mid T_2 \rrbracket &= \llbracket T_1 \rrbracket \text{ or } \llbracket T_2 \rrbracket \\
\llbracket \text{any}() \rrbracket &= \text{term}() \\
\llbracket \text{none}() \rrbracket &= \text{none}() \\
\llbracket \text{atom}() \rrbracket &= \text{atom}() \\
\llbracket :k \rrbracket &= :k \\
\llbracket \text{pid}() \rrbracket &= \text{pid}() \\
\llbracket \text{port}() \rrbracket &= \text{port}() \\
\llbracket \text{reference}() \rrbracket &= \text{reference}() \\
\llbracket \text{float}() \rrbracket &= \text{float}() \\
\llbracket \text{integer}() \rrbracket &= \text{integer}() \\
\llbracket \text{neg_integer} \rrbracket &= \text{infty}..-1 \\
\llbracket \text{non_neg_integer} \rrbracket &= 0..\text{infty} \\
\llbracket \text{pos_integer} \rrbracket &= 1..\text{infty} \\
\llbracket \text{tuple}() \rrbracket &= \text{tuple}() \\
\llbracket \{\bar{T}\} \rrbracket &= \{\llbracket \bar{T} \rrbracket\} \\
\llbracket \% \{F_1, \dots, F_n\} \rrbracket &= \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket) \\
\llbracket [] \rrbracket &= \text{empty_list}() \\
\llbracket \text{nonempty_maybe_improper_list}(T_1, T_2) \rrbracket &= \text{non_empty_list}(\llbracket T_1 \rrbracket, \llbracket T_2 \rrbracket) \\
\llbracket \langle\!\langle \rangle\!\rangle \rrbracket &= \text{bitstring}() \\
\llbracket \langle\!_::n\!\rangle \rrbracket &= \begin{cases} \text{binary}() & \text{if } n \bmod 8 = 0 \\ \text{bitstring}() & \text{otherwise} \end{cases} \\
\llbracket \langle\!_::_*n\!\rangle \rrbracket &= \begin{cases} \text{binary}() & \text{if } n \bmod 8 = 0 \\ \text{bitstring}() & \text{otherwise} \end{cases} \\
\llbracket \langle\!_::n, _::_*n\!\rangle \rrbracket &= \begin{cases} \text{binary}() & \text{if } n \bmod 8 = 0 \\ \text{bitstring}() & \text{otherwise} \end{cases} \\
\llbracket (\bar{T} \rightarrow T) \rrbracket &= (\llbracket \bar{T} \rrbracket \rightarrow \llbracket T \rrbracket) \\
\llbracket (\dots \rightarrow T) \rrbracket &= \begin{cases} \text{fun}() & \text{if } \llbracket T \rrbracket \approx \text{term}() \\ \text{undefined} & \text{otherwise} \end{cases} \\
\llbracket n \rrbracket &= n..n \\
\llbracket n..n' \rrbracket &= n..n'
\end{aligned}$$

Figure 5: Translation in Types

3.3 Translation of Fields

Let τ range over the right-hand side type of field types, that is, τ is either T or $\text{if_set}(T)$. We consider lists whose elements are of the form $K \Rightarrow \tau$. Given such a list L we write $K \in L$ if there exist τ such that the element $K \Rightarrow \tau$ belongs to L . We denote by Empty the lists that maps all the non-singleton key types to $\text{not_set}()$.

First we define the translation of single field types. If $K_1, \dots, K_n$ are pairwise distinct key types, we write $[\bigvee_{i=1}^n K_i \Rightarrow \tau]$ for the list $[K_1 \Rightarrow \tau, \dots, K_n \Rightarrow \tau]$. Then the translation of single field types is defined in Figure 6:

$$\llbracket \text{required}(\mathbf{T}) \Rightarrow \mathbf{T}' \rrbracket = \begin{cases} \llbracket :k \Rightarrow \llbracket \mathbf{T}' \rrbracket \rrbracket & \text{if } \llbracket \mathbf{T} \rrbracket \approx :k \\ \llbracket \bigvee_{i \in I} \mathbf{K}_i \Rightarrow \text{if_set}(\llbracket \mathbf{T}' \rrbracket) \rrbracket & \text{if } \llbracket \mathbf{T} \rrbracket \approx \bigvee_{i \in I} \mathbf{K}_i \text{ and } \llbracket \mathbf{T} \rrbracket \not\approx :k \\ \text{undefined} & \text{otherwise} \end{cases}$$

$$\llbracket \text{optional}(\mathbf{T}) \Rightarrow \mathbf{T}' \rrbracket = \begin{cases} \llbracket \bigvee_{i \in I} \mathbf{K}_i \Rightarrow \text{if_set}(\llbracket \mathbf{T}' \rrbracket) \rrbracket & \text{if } \llbracket \mathbf{T} \rrbracket \approx \bigvee_{i \in I} \mathbf{K}_i \\ \text{undefined} & \text{otherwise} \end{cases}$$

where all $\mathbf{K}_i$'s in I are pairwise distinct.

Figure 6: Translation in Field Types

After obtaining types in Elixir Types we can define merge function. Given two field lists L_1 and L_2 , we write $L_1 \oplus L_2$ the list obtained by adding to L_1 all the fields elements $\mathbf{K} \Rightarrow \tau$ of L_2 such that $\mathbf{K} \notin L_1$ and $\mathbf{K} \neq :k$. Formally:

$$\begin{aligned} L_1 \oplus [] &= L_1 \\ L_1 \oplus [\mathbf{K} \Rightarrow \tau, L_2] &= L_1 \oplus L_2 && \text{if } \mathbf{K} \in L_1 \\ L_1 \oplus [\mathbf{K} \Rightarrow \tau, L_2] &= [\mathbf{K} \Rightarrow \tau, L_1] \oplus L_2 && \text{if } \mathbf{K} \notin L_1 \\ L_1 \oplus [:k \Rightarrow \tau, L_2] &= L_1 \oplus L_2 && \text{if } \mathbf{K} \in L_1 \text{ and } \mathbf{K} \neq :k \\ L_1 \oplus [\mathbf{K} \Rightarrow \tau, L_2] &= [\mathbf{K} \text{ and not } :k \Rightarrow \tau, L_1] \oplus L_2 && \text{if } :k \in L_1 \text{ and } \mathbf{K} \neq :k \\ L_1 \oplus L_2 &= \text{undefined} && \text{otherwise} \end{aligned}$$

Figure 7: Merge in Field Types

Notice that merging naturally reverses the order of field types, that is because field types in TypeSpecs are left-associative whereas those in Elixir Types are right-associative.

We then finally define $\text{map}(L)$ as the map type whose field list is L . Then we have $\llbracket \% \{F_1, \dots, F_n\} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket)$. If we want a saturated map type (i.e., a map type that specifies all the possible key types), then we have $\llbracket \% \{F_1, \dots, F_n\} \rrbracket = \text{map}(\bigoplus_{i=1}^n \llbracket F_i \rrbracket \oplus \text{Empty})$.

In spite of the whole translation process we have completed, there are two missing parts to be complemented: merging of field types with their left-hand side undefined and that being union types. Even though they share the nature of joint sets with `atom()` and `:k`, the former of to-be-approximated is non-existential in key types of Elixir Types and the latter has to be treated one by one within its union(s). Therefore, the proceeding section extends this merge function for the undefined.

4 Total Approximating Translation of TypeSpecs into Elixir Types

The translation defined so far is partial since some TypeSpecs types do not have a corresponding type in Elixir Types. This is not a weakness of the Elixir Types system, but rather a consequence of the design choices made for Elixir Types, to keep them as simple as possible while still being useful. However, it is possible to define a total translation from types in TypeSpecs to ones in Elixir Types hence approximating the original types in TypeSpecs. This total translation can be useful in some contexts, for example, when we want to ensure that every type in TypeSpecs can be represented in Elixir Types even approximately.

More precisely, the previous translations are undefined in the following cases:

- $\llbracket (\dots \rightarrow \mathbf{T}) \rrbracket$: the top function type when $\llbracket \mathbf{T} \rrbracket$ is not equivalent to `any()`.
- $\llbracket \text{required}(\mathbf{T}) \Rightarrow \mathbf{T}' \rrbracket$ and $\llbracket \text{optional}(\mathbf{T}) \Rightarrow \mathbf{T}' \rrbracket$: the field types when $\llbracket \mathbf{T} \rrbracket$ is equivalent to a singleton atom type or is not equivalent to a disjunction of pairwise distinct key types.

In the rest of this section we give a conservative extension of the translation defined so far that is total, i.e., it is defined for every type in TypeSpecs. The extension consists in defining approximations for the cases where the previous translation is undefined.

4.1 Approximation of Top Function Types

The top function type is a set of all the possible functions, that is, unions of functions returning the top type with every arity available as defined in Figure 8.

$$\dots \rightarrow \text{any}() \equiv \text{none}() \rightarrow \text{any}() \mid \dots \mid \underbrace{(\text{none}(), \dots, \text{none}()) \rightarrow \text{any}()}_{255 \text{ times}}$$

Figure 8: Top Function Type

Elixir Types provides the semantically same type, `fun()`, which enables the direct translation for `... -> any()`, although, Elixir Types does not directly define `fun()` as Figure 8, due to the triviality of defining the list of functions with all the possible arity, and the cost of defining and having all the function types which expand to maximum arity of 255. Nonetheless, the fundamental difference in allowance from TypeSpecs, i.e., letting to specify the codomain of a function type, brings subtypes of the top function type on approximation.

The approximation is rather simple because the leniency in expressiveness is proven to be trivial in practice for several reasons. Most of all, Dialyzer does not differentiate the return type and perceive any form of the top types as `... -> any()`. Which means, in TypeSpecs with Dialyzer, `... -> any()` can replace any other top type functions with a codomain different than `any()` and the same applies to the other way around. Secondly, one of the concerns with moving forward with Elixir Types is to fail typecheck, but typecheck will trivially succeed even alone with `fun()` when approximated since the top function type already takes over all the other function types. One small problem is that this approximation will blunt the specification, hence we can write `fun()` and `dynamic()`, which is a subtype of `fun()`, to semantically display that the approximation is a subtype of the top function type.

$$\llbracket (\dots \rightarrow T) \rrbracket \approx \begin{cases} \text{fun}() & \text{if } \llbracket T \rrbracket \approx \text{term}() \\ \text{fun}() \text{ and } \text{dynamic}() & \text{otherwise} \end{cases}$$

Figure 9: Approximation of Top Function Type

4.2 Approximation of Field Types

The merging case with disjoint left-hand sides does not only perform merge function but also conduct approximation of types in TypeSpecs that are smaller than key types in Elixir Types. That is because merge must take original intention of types in TypeSpecs into account, but if we compare types after approximation, in particular over-approximation, users' original aim could be deviated. Ultimately, approximated merging involves two cases: firstly, when a left-hand side type in TypeSpecs is smaller than any key types in Elixir Types; secondly, when it involves a union type.

To begin with defining the first case, let s represent any types that are smaller than key types in Elixir Types. With that said, an s becomes a subtype of a K from Elixir Types, i.e., $s \leq K$, e.g., `1..3` is a subtype of `integer()`. In addition, we define K^s to describe when s is promoted to their supertypes, and τ_s to indicate the right-hand side type of K^s in L_1 . In extension to the merge function in Figure 7:

$$\begin{aligned} [s \Rightarrow \tau] \oplus [] &= [K^s \Rightarrow \tau] \\ L_1 \oplus [s \Rightarrow \tau, L_2] &= L_1 \oplus L_2 && \text{if } K \in L_1 \\ L_1 \oplus [s \Rightarrow \tau, L_2] &= [K^s \Rightarrow \tau, L_1] \oplus L_2 && \text{if } K \notin L_1 \\ L_1 \oplus [K \Rightarrow \tau, L_2] &= [K \Rightarrow \tau \text{ or } \tau_s, L_1 \text{ and not } K^s \Rightarrow \tau_s] \oplus L_2 && \text{if } K^s \in L_1 \end{aligned}$$

Figure 10: Approximated Merge in Field Types for Singletons

As you can see, the reason why we distinguish the promoted types from those not is to leave the intention of using the singleton types in TypeSpecs while maintaining the soundness.

This works, for instance, when attempted merging `1..3 => float()` and `integer() => integer()`. The first merge occurs with `1..3 => float()` and an empty list which is the first case of every merge, which produces `integer()s => float()s`. Consecutively, it merges with `integer() => integer()`. This results in `integer() => float()s or integer()`. Trivially, keep in mind that the superscript s does not appear in the final result for users.

Secondly, approximation for union types as key types could be defined in the similar way. And yet we do not specify extra approximation because most of the parts with union types are translated with the basic translation.

Which merely leave us to define approximation when union types partially overlaps. For this special case, let U be a key type or union types, and let u be a type that is joint with U , hence $U = \bigcap_{i \in I} u_i$.

$$L_1 \oplus [U \Rightarrow \tau, L_2] = [U \text{ and not } u \Rightarrow \tau, L_1] \oplus L_2 \quad \text{if } u \in L_1$$

Figure 11: Approximated Merge in Field Types for Unions

To illustrate, take a union type like `pid() => binary()`, which is in L_1 , to merge it with `pid() | port() | reference() => atom()`. This results in `[port() | reference() => atom(), pid() => binary()]` by the definition.

There may be a few views that this seems verbose to break approximated merge function into two cases. Nevertheless, Eventually the facts that necessitated this approximation are the subtle differences that come from the need of type promotion for singleton types and that of clear definition for partial joint union types. Which, therefore, have been addressed clearly so far.